DATABASE ASSIGNMENT TASKS PRACTICE

```
CREATE database FoodDeliviery; USE FoodDeliviery;
CREATE table customers(
id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100),
email VARCHAR(100) UNIQUE,
signup_date DATE
);
CREATE TABLE restaurants (
id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100),
cuisine VARCHAR(50)
);
CREATE TABLE orders (
id INT PRIMARY KEY AUTO_INCREMENT,
customer_id INT, restaurant_id INT, total DECIMAL(10,2),
status VARCHAR(30), created_at DATETIME,

FOREIGN KEY(customer_id) REFERENCES customers(id), FOREIGN KEY(restaurant_id)
REFERENCES restaurants(id)
);
CREATE TABLE accounts (
id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50),
balance DECIMAL(10,2)
);
```

Customers:

INSERT INTO customers (name, email, signup_date) VALUES

('Pavan','pavan@gmail.com','2025-01-05'),

('Viraj','viraj@gmail.com','2025-01-08'),

('Param','param@gmail.com','2025-01-12'),

('Om','om@gmail.com','2025-01-18'),

('Khushi','khushi@gmail.com','2025-01-25'),

('Riya','riya@gmail.com','2025-02-01'),

('Komal','komal@gmail.com','2025-02-06'),

('Aashesh','aashesh@gmail.com','2025-02-12'),

('Jiya','jiya@gmail.com','2025-02-20'),

('Manan','manan@gmail.com','2025-02-25'),

('Swethin','swethin@gmail.com','2025-03-01'),

('Aastha','aastha@gmail.com','2025-03-06'),

('Kanishk','kanishk@gmail.com','2025-03-10'),

('Pranjal','pranjal@gmail.com','2025-03-15'),

('Aryan','aryan@gmail.com','2025-03-20'),

('Poojan','poojan@gmail.com','2025-03-25'),

('Kunj','kunj@gmail.com','2025-04-02'),

('Ronak','ronak@gmail.com','2025-04-07'),

('Alice','alice@gmail.com','2025-04-15'),

('Bob','bob@gmail.com','2025-04-20');

Restaurants:

INSERT INTO restaurants (name, cuisine) VALUES

('La Pino\'z Pizza','Pizza'),

('Burger King','Burger'),

('DFS Momos','Momos'),

('Agnee Bakery','Sandwich'),

('Frankie Station','Frankie'),

('Cafe Banana','Noodles'),

('Spicy Speculation','Chap'),

('Nihao Truck','Sushi'),

('Barista','Pasta'),

('Coffee Culture','Croissants');

Orders:

INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES

(1,1,450.50,'Completed','2025-05-01 12:00:00'),

(1,3,320.00,'Completed','2025-05-05 14:00:00'),

(2,2,280.00,'Pending','2025-05-02 13:00:00'),

(2,5,600.00,'Completed','2025-05-08 18:00:00'),

(3,4,390.00,'Cancelled','2025-05-03 16:00:00'),

(3,1,250.00,'Completed','2025-05-10 19:00:00'),

(4,7,550.00,'Completed','2025-05-04 20:00:00'),

(4,3,450.00,'Pending','2025-05-09 11:30:00'),

(5,8,700.00,'Completed','2025-05-06 17:00:00'),

(5,2,260.00,'Completed','2025-05-12 18:20:00'),

(6,6,800.00,'Completed','2025-05-07 13:30:00'),

(6,9,950.00,'Pending','2025-05-15 20:30:00'),

(7,10,350.00,'Completed','2025-05-08 09:20:00'),

(7,5,420.00,'Completed','2025-05-18 12:15:00'),

(8,4,500.00,'Cancelled','2025-05-09 15:10:00'),

(8,1,620.00,'Completed','2025-05-20 18:45:00'),

(9,3,710.00,'Pending','2025-05-11 11:00:00'),

(9,8,380.00,'Completed','2025-05-22 17:30:00'),

(10,2,450.00,'Completed','2025-05-12 19:20:00'),

(10,6,900.00,'Completed','2025-05-24 21:00:00'),

(11,7,520.00,'Pending','2025-05-13 10:00:00'),

(11,4,610.00,'Completed','2025-05-26 13:00:00'),

(12,5,450.00,'Completed','2025-05-14 14:00:00'),

(12,8,780.00,'Completed','2025-05-28 18:00:00'),

(13,9,840.00,'Pending','2025-05-15 16:00:00'),

(13,1,370.00,'Completed','2025-05-29 20:00:00'),

(14,2,490.00,'Completed','2025-05-16 12:00:00'),

(14,6,860.00,'Cancelled','2025-05-30 19:00:00'),

(15,3,540.00,'Completed','2025-05-17 15:00:00'),

(15,10,310.00,'Completed','2025-06-01 18:00:00'),

(16,7,650.00,'Pending','2025-05-18 17:00:00'),

(16,5,290.00,'Completed','2025-06-02 20:00:00'),

(17,8,720.00,'Completed','2025-05-19 11:00:00'),

(17,9,900.00,'Completed','2025-06-03 13:00:00'),

(18,1,470.00,'Completed','2025-05-20 14:30:00'),

(18,2,350.00,'Pending','2025-06-04 19:30:00'),

(19,4,510.00,'Completed','2025-05-21 10:15:00'),

(19,6,680.00,'Completed','2025-06-05 16:30:00'),

(20,3,450.00,'Completed','2025-05-22 12:30:00'),

(20,7,760.00,'Cancelled','2025-06-06 18:30:00'),

(1,5,550.00,'Completed','2025-06-07 12:00:00'),

(2,8,620.00,'Completed','2025-06-08 14:00:00'),

(3,9,710.00,'Pending','2025-06-09 16:00:00'),

(4,10,450.00,'Completed','2025-06-10 18:00:00'),

(5,6,830.00,'Completed','2025-06-11 20:00:00'),

(6,2,310.00,'Completed','2025-06-12 13:00:00'),

(7,4,480.00,'Completed','2025-06-13 15:00:00'),

(8,7,690.00,'Pending','2025-06-14 17:00:00'),

(9,1,520.00,'Completed','2025-06-15 19:00:00'),

(10,5,410.00,'Completed','2025-06-16 21:00:00');

Accounts:

INSERT INTO accounts(name, balance) VALUES

('Alice', 699.00),

('Bob', 799.00);

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

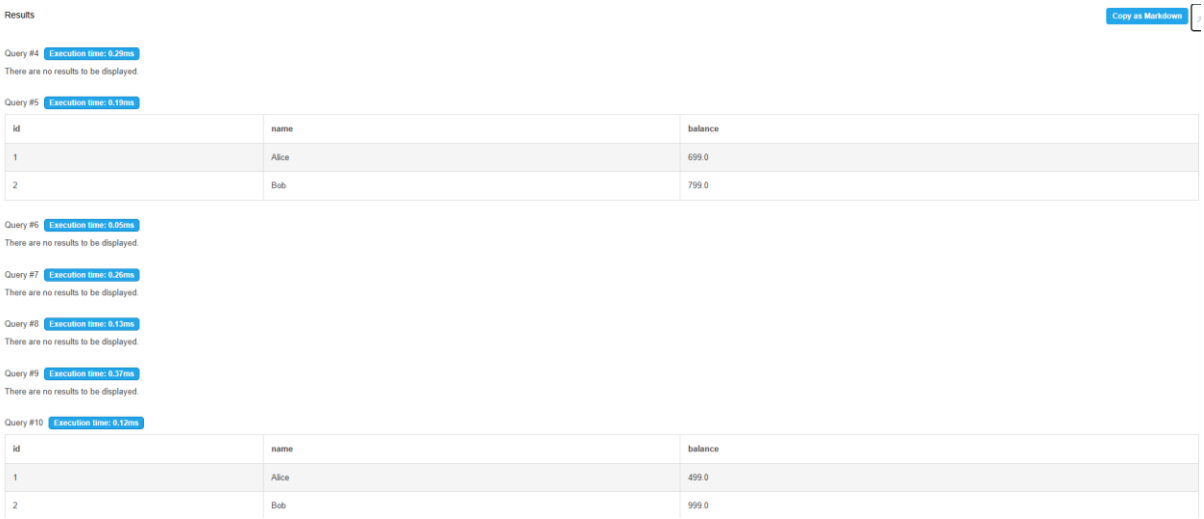
b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

TASK (a and b)

Query SQL

```sql
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name='Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name='Bob';

ROLLBACK;

SELECT * FROM accounts;

START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE ID =1;

UPDATE accounts
SET balance = balance + 200
WHERE ID =2;
COMMIT;

SELECT * FROM accounts;
```



TASK c

Atomicity is the ACID property that guarantees the money is never "lost" in the middle of a transfer. If any part of the transaction fails or is aborted via a ROLLBACK, the entire sequence is undone, restoring the system to its original state.

TASK 2 Joins

Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

a) Write an INNER JOIN that lists every customer who has placed at least one order.

```
27
28 SELECT c.name, o.id,o.total
29 FROM customers c
30 INNER JOIN orders o
31 ON c.id=o.customer_id;
32
```

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

```
34
35 FROM customers c
36 LEFT JOIN orders o
37 ON c.id=o.customer_id;
38
```

```
39 SELECT c.id, c.name
40 FROM customers c
41 LEFT JOin orders o
42 ON c.id=o.customer_id
43 WHERE o.id IS NULL;
44
```

c)In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

A marketing team prefers a LEFT JOIN over an INNER JOIN because it shows user or customers who have signed up but never ordered. They can target them to give discounts and special offers so that they can make their first purchase.

INNER JOIN:

Results

Copy as Markdown

name	id	total
Pavan	1	450.5
Pavan	2	320.0
Pavan	41	550.0
Viraj	3	280.0
Viraj	4	600.0
Viraj	42	620.0
Param	5	390.0
Param	6	250.0
Param	43	710.0
Om	7	550.0
Om	8	450.0
Om	44	450.0
Khushi	9	700.0
Khushi	10	260.0
Khushi	45	830.0
Riya	11	800.0
Riya	12	950.0
Riya	46	310.0
Komal	13	350.0
Komal	14	420.0
Komal	47	480.0
Aashesh	15	500.0
Aashesh	16	620.0
Aashesh	48	690.0
Jiya	17	710.0
Jiya	18	380.0
Jiya	49	520.0
Manan	19	450.0
Manan	20	900.0
Manan	50	410.0
Swethin	21	520.0
Swethin	22	610.0
Aastha	23	450.0
Aastha	24	780.0
Kanishk	25	840.0
Kanishk	26	370.0
Pranjal	27	490.0
Pranjal	28	860.0
Aryan	29	540.0
Aryan	30	310.0
Poojan	31	650.0
Poojan	32	290.0
Kunj	33	720.0
Kunj	34	900.0
Ronak	35	470.0
Ronak	36	350.0
Alice	37	510.0
Alice	38	680.0
Bob	39	450.0
Bob	40	760.0

LEFT JOIN:

Query #12 Execution time: 0.27ms

id	name	order_id
1	Pavan	1
1	Pavan	2
1	Pavan	41
2	Viraj	3
2	Viraj	4
2	Viraj	42
3	Param	5
3	Param	6
3	Param	43
4	Om	7
4	Om	8
4	Om	44
5	Khushi	9
5	Khushi	10
5	Khushi	45
6	Riya	11
6	Riya	12
6	Riya	46
7	Komal	13
7	Komal	14
7	Komal	47
8	Aashesh	15
8	Aashesh	16
8	Aashesh	48
9	Jiya	17
9	Jiya	18
9	Jiya	49
10	Manan	19
10	Manan	20
10	Manan	50
11	Swethin	21
11	Swethin	22
12	Aastha	23
12	Aastha	24
13	Kanishk	25
13	Kanishk	26
14	Pranjal	27
14	Pranjal	28
15	Aryan	29
15	Aryan	30
16	Poojan	31
16	Poojan	32
17	Kunj	33
17	Kunj	34
18	Ronak	35
18	Ronak	36
19	Alice	37
19	Alice	38
20	Bob	39
20	Bob	40

Query #13 Execution time: 0.16ms

id	name

There are no results to be displayed.

TASK 3 Indexing

Using your orders table:

a) Pick a column that is not currently indexed (for example status or created_at).

b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

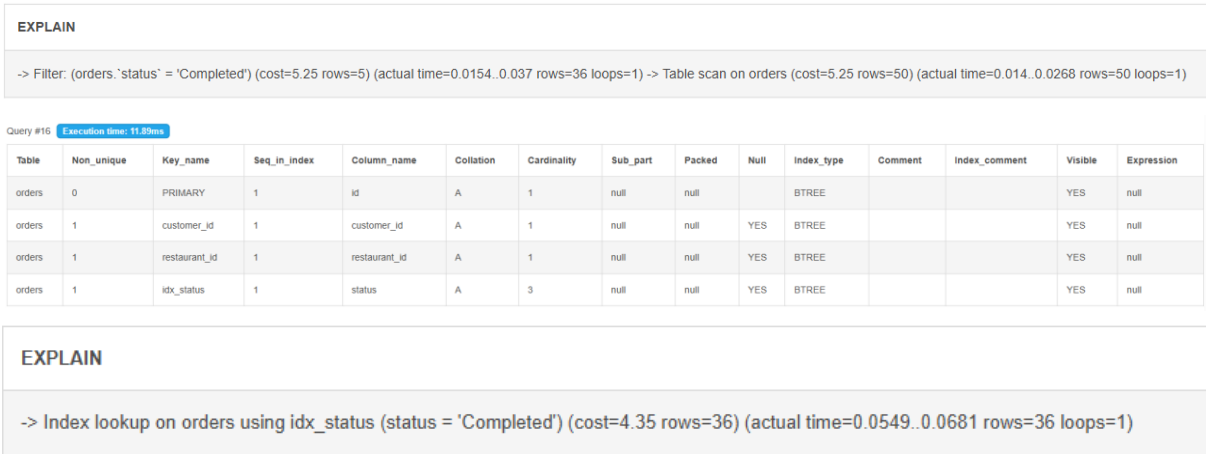
c) Create an index on that column.

d) Re-run the exact same query and record the new numbers.

e) Submit a one-line comparison of before vs. after.

TASK (a, b, c, d)

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status='Completed';

CREATE INDEX idx_status
ON orders(status);
SHOW INDEX FROM orders;

CREATE INDEX idx_status
ON orders(status);
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE status='Completed';
```



EXPLAIN

-> Filter: (orders.`status` = 'Completed') (cost=5.25 rows=5) (actual time=0.0154..0.037 rows=36 loops=1) -> Table scan on orders (cost=5.25 rows=50) (actual time=0.014..0.0268 rows=50 loops=1)

Query #16 Execution time: 11.89ms

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	id	A	1	null	null		BTREE			YES	null
orders	1	customer_id	1	customer_id	A	1	null	null	YES	BTREE			YES	null
orders	1	restaurant_id	1	restaurant_id	A	1	null	null	YES	BTREE			YES	null
orders	1	idx_status	1	status	A	3	null	null	YES	BTREE			YES	null

EXPLAIN

-> Index lookup on orders using idx_status (status = 'Completed') (cost=4.35 rows=36) (actual time=0.0549..0.0681 rows=36 loops=1)

e) In **DB Fiddle**, before creating the index, the query scanned the entire orders table to find rows where status = 'Completed'.

After creating the index (CREATE INDEX idx_status ON orders(status);), the query used the index to find the matching rows directly, making it faster and more efficient.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

SELECT * FROM orders WHERE customer_id IN (

SELECT id FROM customers WHERE signup_date >= 'some date'

);

a) Rewrite it as a JOIN instead of a subquery.

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

```
59
60 SELECT * FROM orders
61 WHERE customer_id IN
62 ( SELECT id
63  FROM customers
64  WHERE signup_date>='2025-01-01'
65  );
66
67 SELECT o.id, o.total, o.status
68 FROM orders o
69 JOIN customers c
70 ON o.customer_id=c.id
71 WHERE c.signup_date>='2025-01-01';
72
73 SELECT * FROM orders
74 WHERE customer_id IN
75 ( SELECT id
76 FROM customers
77 WHERE signup_date>='2025-01-01'
78 );
79
80 SELECT o.id, o.total, o.status
81 FROM orders o
82 JOIN customers c
83 on o.customer_id=c.id
84 WHERE c.signup_date>='2025-01-01';
85
```

a) a) Rewrite it as a JOIN instead of a subquery.

Query #18 Execution time: 0.42ms

id	customer_id	restaurant_id	total	status	created_at
1	1	1	485.0	Completed	2026-04-01 12:00:00
2	1	3	320.0	Completed	2026-04-05 14:00:00
3	2	2	260.0	Pending	2026-04-02 13:00:00
4	2	5	600.0	Completed	2026-04-08 19:00:00
5	3	4	390.0	Cancelled	2026-04-03 10:00:00
6	3	1	285.0	Completed	2026-04-10 18:00:00
7	4	7	560.0	Completed	2026-04-04 20:00:00
8	4	3	450.0	Pending	2026-04-09 11:30:00
9	5	8	720.0	Completed	2026-04-06 17:00:00
10	5	2	310.0	Completed	2026-04-12 10:30:00
11	6	6	405.0	Completed	2026-04-07 13:30:00
12	6	9	980.0	Pending	2026-04-15 20:30:00
13	7	10	340.0	Completed	2026-04-09 09:30:00
14	7	8	425.0	Completed	2026-04-16 12:15:00
15	8	4	600.0	Cancelled	2026-04-09 15:10:00
16	8	1	620.0	Completed	2026-04-20 18:45:00
17	9	3	715.0	Pending	2026-04-11 11:00:00
18	9	5	360.0	Completed	2026-04-22 14:30:00
19	10	3	455.0	Completed	2026-04-13 19:00:00
20	10	6	900.0	Completed	2026-04-24 21:00:00
21	11	7	520.0	Pending	2026-04-13 10:00:00
22	11	4	610.0	Completed	2026-04-26 13:00:00
23	12	9	480.0	Completed	2026-04-14 18:00:00
24	12	8	780.0	Completed	2026-04-28 10:00:00
25	13	9	840.0	Pending	2026-04-15 16:00:00
26	13	1	370.0	Completed	2026-04-29 20:00:00
27	14	2	490.0	Completed	2026-04-16 12:00:00
28	14	6	660.0	Cancelled	2026-05-30 19:00:00
29	15	5	540.0	Completed	2026-05-17 15:00:00
30	15	10	310.0	Completed	2026-05-01 18:00:00
31	16	7	650.0	Pending	2026-05-18 17:00:00
32	16	5	390.0	Completed	2026-05-02 20:00:00
33	17	8	720.0	Completed	2026-05-19 11:00:00
34	17	9	900.0	Completed	2026-05-03 13:00:00
35	18	1	470.0	Completed	2026-05-20 14:30:00
36	18	2	350.0	Pending	2026-05-04 19:30:00
37	19	4	510.0	Completed	2026-05-21 10:15:00
38	19	6	680.0	Completed	2026-05-05 16:30:00
39	20	3	450.0	Completed	2026-05-22 12:30:00
40	20	7	760.0	Cancelled	2026-05-06 18:30:00
41	1	5	550.0	Completed	2026-05-07 12:00:00
42	2	8	620.0	Completed	2026-05-08 14:00:00
43	3	9	710.0	Pending	2026-05-09 16:00:00
44	4	10	480.0	Completed	2026-05-10 18:00:00
45	5	6	830.0	Completed	2026-05-11 20:00:00
46	6	2	310.0	Completed	2026-05-12 13:00:00
47	7	4	480.0	Completed	2026-05-13 15:00:00
48	8	7	690.0	Pending	2026-05-14 17:00:00
49	9	1	620.0	Completed	2026-05-15 19:00:00
50	10	5	410.0	Completed	2026-05-16 21:00:00

Query #19 Execution time: 0.25ms

id	total	status
1	485.0	Completed
2	320.0	Completed
3	260.0	Pending
4	600.0	Completed
5	390.0	Cancelled
6	285.0	Completed
7	560.0	Completed
8	450.0	Pending
9	720.0	Completed
10	310.0	Completed
11	405.0	Completed
12	980.0	Pending
13	340.0	Completed
14	425.0	Completed
15	600.0	Cancelled
16	620.0	Completed
17	715.0	Pending
18	360.0	Completed
19	455.0	Completed
20	900.0	Completed
21	520.0	Pending
22	610.0	Completed
23	480.0	Completed
24	780.0	Completed
25	840.0	Pending
26	370.0	Completed
27	490.0	Completed
28	660.0	Cancelled
29	540.0	Completed
30	310.0	Completed
31	650.0	Pending
32	390.0	Completed
33	720.0	Completed
34	900.0	Completed
35	470.0	Completed
36	350.0	Pending
37	510.0	Completed
38	680.0	Completed
39	450.0	Completed
40	760.0	Cancelled
41	550.0	Completed
42	620.0	Completed
43	710.0	Pending
44	480.0	Completed
45	830.0	Completed
46	310.0	Completed
47	480.0	Completed
48	690.0	Pending
49	620.0	Completed
50	410.0	Completed

b.) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

id	customer_id	restaurant_id	total	status	created_at
1	1	1	450.5	Completed	2025-05-01 12:00:00
2	1	3	320.0	Completed	2025-05-05 14:00:00
3	2	2	280.0	Pending	2025-05-02 13:00:00
4	2	5	600.0	Completed	2025-05-08 18:00:00
5	3	4	390.0	Cancelled	2025-05-03 16:00:00
6	3	1	250.0	Completed	2025-05-10 19:00:00
7	4	7	550.0	Completed	2025-05-04 20:00:00
8	4	3	450.0	Pending	2025-05-09 11:30:00
9	5	8	700.0	Completed	2025-05-06 17:00:00
10	5	2	260.0	Completed	2025-05-12 18:20:00
11	6	6	800.0	Completed	2025-05-07 13:30:00
12	6	9	960.0	Pending	2025-05-15 20:30:00
13	7	10	350.0	Completed	2025-05-08 09:20:00
14	7	5	420.0	Completed	2025-05-18 12:15:00
15	8	4	500.0	Cancelled	2025-05-09 15:10:00
16	8	1	620.0	Completed	2025-05-20 18:45:00
17	9	3	710.0	Pending	2025-05-11 11:00:00
18	9	8	380.0	Completed	2025-05-22 17:30:00
19	10	2	450.0	Completed	2025-05-12 19:20:00
20	10	6	900.0	Completed	2025-05-24 21:00:00
21	11	7	520.0	Pending	2025-05-13 10:00:00
22	11	4	610.0	Completed	2025-05-26 13:00:00
23	12	5	450.0	Completed	2025-05-14 14:00:00
24	12	8	780.0	Completed	2025-05-28 18:00:00
25	13	9	840.0	Pending	2025-05-15 16:00:00
26	13	1	370.0	Completed	2025-05-29 20:00:00
27	14	2	490.0	Completed	2025-05-16 12:00:00
28	14	6	860.0	Cancelled	2025-05-30 19:00:00
29	15	3	540.0	Completed	2025-05-17 15:00:00
30	15	10	310.0	Completed	2025-06-01 18:00:00
31	16	7	650.0	Pending	2025-05-18 17:00:00
32	16	5	290.0	Completed	2025-06-02 20:00:00
33	17	8	720.0	Completed	2025-05-19 11:00:00
34	17	9	900.0	Completed	2025-06-03 13:00:00
35	18	1	470.0	Completed	2025-05-20 14:30:00
36	18	2	350.0	Pending	2025-06-04 19:30:00
37	19	4	510.0	Completed	2025-05-21 10:15:00
38	19	6	680.0	Completed	2025-06-05 16:30:00
39	20	3	450.0	Completed	2025-05-22 12:30:00
40	20	7	760.0	Cancelled	2025-06-06 18:30:00
41	1	5	550.0	Completed	2025-06-07 12:00:00
42	2	8	620.0	Completed	2025-06-08 14:00:00
43	3	9	710.0	Pending	2025-06-09 16:00:00
44	4	10	450.0	Completed	2025-06-10 18:00:00
45	5	6	830.0	Completed	2025-06-11 20:00:00
46	6	2	310.0	Completed	2025-06-12 13:00:00
47	7	4	480.0	Completed	2025-06-13 15:00:00
48	8	7	690.0	Pending	2025-06-14 17:00:00
49	9	1	520.0	Completed	2025-06-15 19:00:00
50	10	5	410.0	Completed	2025-06-16 21:00:00

c.) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Results

Copy as Markdown

id	total	status
1	450.5	Completed
2	320.0	Completed
3	280.0	Pending
4	600.0	Completed
5	390.0	Cancelled
6	250.0	Completed
7	550.0	Completed
8	450.0	Pending
9	700.0	Completed
10	260.0	Completed
11	800.0	Completed
12	950.0	Pending
13	350.0	Completed
14	420.0	Completed
15	500.0	Cancelled
16	620.0	Completed
17	710.0	Pending
18	380.0	Completed
19	450.0	Completed
20	900.0	Completed
21	520.0	Pending
21	520.0	Pending
22	610.0	Completed
23	450.0	Completed
24	780.0	Completed
25	840.0	Pending
26	370.0	Completed
27	490.0	Completed
28	860.0	Cancelled
29	540.0	Completed
30	310.0	Completed
31	650.0	Pending
32	290.0	Completed
33	720.0	Completed
34	900.0	Completed
35	470.0	Completed
36	350.0	Pending
37	570.0	Completed
38	680.0	Completed
39	450.0	Completed
40	760.0	Cancelled
41	550.0	Completed
42	620.0	Completed
43	710.0	Pending
44	450.0	Completed
45	830.0	Completed
46	310.0	Completed
47	480.0	Completed
48	640.0	Pending
49	520.0	Completed
50	410.0	Completed

PART 3

Concepts used: *this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.*

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

- a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.
- b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.
- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.
- d) Re-run the same query and compare the new numbers to step (b).
- e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.
- f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

```
86
87 SELECT r.name AS Restaurant,
88 o.total, o.status, o.created_at
89 FROM orders o
90 JOIN restaurants r
91 ON o.restaurant_id=r.id
92 WHERE o.customer_id=5
93 ORDER BY o.created_at DESC
94 LIMIT 5;
```

Query #22 Execution time: 0.28ms

Restaurant	total	status	created_at
Cafe Banana	830.0	Completed	2025-06-11 20:00:00
Burger King	260.0	Completed	2025-05-12 18:20:00
Nihao Truck	700.0	Completed	2025-05-06 17:00:00

b)

```
97 SELECT r.name, o.total, o.status, o.created_at
98 FROM orders o
99 JOIN restaurants r
100 ON o.restaurant_id=r.id
101 WHERE o.customer_id=5
102 ORDER BY o.created_at DESC
103 LIMIT 5;
```

Query #22 Execution time: 0.28ms

Restaurant	total	status	created_at
Cafe Banana	830.0	Completed	2025-06-11 20:00:00
Burger King	260.0	Completed	2025-05-12 18:20:00
Nihao Truck	700.0	Completed	2025-05-06 17:00:00

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Query #24 Execution time: 17.52ms

There are no results to be displayed.

d.) Re-run the same query and compare the new numbers to step (b).

```
SELECT r.name AS Restaurant,
o.total, o.status, o.created_at
FROM orders o
JOIN restaurants r
ON o.restaurant_id=r.id
WHERE o.customer_id=5
ORDER BY o.created_at DESC
LIMIT 5;
```

Query #25 Execution time: 0.43ms

Restaurant	total	status	created_at
Cafe Banana	830.0	Completed	2025-06-11 20:00:00
Burger King	260.0	Completed	2025-05-12 18:20:00
Nihao Truck	700.0	Completed	2025-05-06 17:00:00

e.) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

```
117 SELECT r.name, o.total, o.status, o.created_at
118 FROM orders o
119 JOIN restaurants r
120 ON o.restaurant_id=r.id
121 WHERE o.customer_id=5
122 ORDER BY o.created_at DESC
123 LIMIT 5;
```

Query #26 Execution time: 0.25ms

name	total	status	created_at
Cafe Banana	830.0	Completed	2025-06-11 20:00:00
Burger King	260.0	Completed	2025-05-12 18:20:00
Nihao Truck	700.0	Completed	2025-05-06 17:00:00

- f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

After creating the index, the database searched fewer rows and found the required records much faster. It also avoided extra sorting, making the query more efficient.

The index idx_customer_orders_date(customer_id, created_at DESC) matches the query. It first finds all orders for customer_id = 5 and then uses the records already sorted by created_at in descending order.